

LABORATORY RECORD

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	4
Student Name	K. Anusha
Roll No.	A24126552088
Date	29-08-2026

1. TITLE

Dynamic Student Registration Form using HTML, CSS and JavaScript (DOM Manipulation and Event Handling)

2. AIM

To design and develop a dynamic Student Registration Form using HTML, CSS, and JavaScript that allows a user to add and delete student records in a table using DOM manipulation and event handling, without reloading the page.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of the Document Object Model (DOM) in JavaScript.
 - To select HTML elements in JavaScript using `getElementById()`.
 - To handle form submission using the `addEventListener()` method.
 - To prevent default form submission behaviour using `event.preventDefault()`.
 - To read and validate values entered in text, number, and select input fields.
 - To dynamically create new HTML elements using `createElement()`.
 - To add dynamically created elements to the page using `appendChild()`.
 - To dynamically add and remove table rows based on user actions.
 - To handle button click events for deleting a specific row.
 - To display dynamic status/validation messages to the user.
-

4. THEORY

The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify. Using the DOM, JavaScript can select existing elements, change their content, and create entirely new elements at runtime, allowing a webpage to update itself dynamically without a page reload.

HTML is used to define the structure of the page, including a registration form and a table to display student records. CSS is used to style the form, buttons, and table. JavaScript is used to add interactivity: it listens for the form's submit event, validates the entered values, and then dynamically builds and inserts a new table row containing the student's details and a Delete button.

In this experiment, a webpage titled **Student Registration** is created using HTML5, styled using an external CSS file named **style.css**, and made interactive using an external JavaScript file named **script.js**.

The webpage consists of:

- **Registration Form:** Contains input fields for Student Name, Roll Number, a Course dropdown, and an "Add Student" submit button.
- **Message Paragraph:** Displays validation errors or success messages such as "Please fill all fields", "Student added successfully", or "Student deleted successfully".
- **Student Table:** Displays the list of registered students with Name, Roll Number, Course, and an Action column containing a Delete button for each row.

JavaScript event handling and DOM manipulation methods such as `addEventListener()`, `createElement()`, `appendChild()`, `textContent`, and `remove()` are used to make the table fully dynamic — rows are added and removed entirely in the browser without a page reload.

The basic working flow is:

HTML Structure → CSS Styling → JavaScript Event Handling → DOM Manipulation → Dynamic Table Update

5. ALGORITHM / PROCEDURE

1. Create a new project folder for the webpage.
2. Create an HTML file named `index.html`.
3. Create an external CSS file named `style.css`.
4. Create an external JavaScript file named `script.js`.
5. Design the page structure with a container `div` and a title heading.
6. Create a registration form with input fields for Student Name and Roll Number, and a dropdown for Course.
7. Add a submit button labelled "Add Student" to the form.
8. Add a paragraph element to display validation and status messages.
9. Create an HTML table with header columns: Name, Roll Number, Course, and Action.
10. Link `style.css` to style the form, table, and buttons.
11. Link `script.js` to add interactivity to the page.
12. In JavaScript, select the form and input elements using `getElementById()`.
13. Add a submit event listener to the form using `addEventListener()`.
14. Prevent the default form submission behaviour using `event.preventDefault()`.
15. Read the values entered in the Name, Roll Number, and Course fields.
16. Validate that all fields are filled; if not, display an error message and stop.
17. Dynamically create a new table row using `document.createElement("tr")`.
18. Create table cells for name, roll number, and course, and set their text using `textContent`.
19. Create a Delete button for the row and add a click event listener to it.
20. Inside the delete handler, remove the row from the table using `row.remove()`.

21. Append the delete button to the action cell, and append all cells to the row.
22. Append the new row to the student table using appendChild().
23. Display a success message and clear the input fields.
24. Open index.html in a web browser and test adding and deleting student records.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>
  <link rel="stylesheet" href="style.css">
</head>

<body>

  <div class="container">

    <h1 id="title">Student Registration Form</h1>

    <form id="studentForm">

      <label>Student Name:</label>
      <input type="text" id="name" placeholder="Enter student name">

      <label>Roll Number:</label>
      <input type="number" id="roll" placeholder="Enter roll number">

      <label>Course:</label>
      <select id="course">
        <option value="">Select Course</option>
        <option value="JavaScript">JavaScript</option>
        <option value="Python">Python</option>
        <option value="Java">Java</option>
      </select>

      <button type="submit">Add Student</button>

    </form>

    <p id="message"></p>

    <h2>Student List</h2>

    <table id="studentTable">

      <thead>
        <tr>
          <th>Name</th>
          <th>Roll Number</th>
          <th>Course</th>
          <th>Action</th>
        </tr>
      </thead>

      <tbody id="studentList">
      </tbody>

    </table>

  </div>

  <script src="script.js"></script>

</body>
</html>
```

style.css

```

body {
  font-family: Arial, sans-serif;
  background-color: #f2f2f2;
}

.container {
  width: 700px;
  margin: 50px auto;
  background-color: white;
  padding: 30px;
  border-radius: 10px;
}

h1 {
  text-align: center;
}

label {
  display: block;
  margin-top: 10px;
}

input,
select {
  width: 100%;
  padding: 10px;
  margin-top: 5px;
  box-sizing: border-box;
}

button {
  margin-top: 15px;
  padding: 10px 20px;
  cursor: pointer;
}

table {
  width: 100%;
  margin-top: 20px;
  border-collapse: collapse;
}

th,
td {
  border: 1px solid black;
  padding: 10px;
  text-align: center;
}

th {
  background-color: #ddd;
}

.deleteBtn {
  background-color: red;
  color: white;
  border: none;
}

```

script.js

```

let form = document.getElementById("studentForm");

let nameInput = document.getElementById("name");
let rollInput = document.getElementById("roll");
let courseInput = document.getElementById("course");

let studentList = document.getElementById("studentList");
let message = document.getElementById("message");

// Form submit event
form.addEventListener("submit", function(event) {

    // Prevent page refresh
    event.preventDefault();

    // Getting values from input fields
    let name = nameInput.value;
    let roll = rollInput.value;
    let course = courseInput.value;

    // Checking empty fields
    if (name === "" || roll === "" || course === "") {
        message.textContent = "Please fill all fields";
        return;
    }

    // Creating a new table row
    let row = document.createElement("tr");

    // Creating table cells
    let nameCell = document.createElement("td");
    let rollCell = document.createElement("td");
    let courseCell = document.createElement("td");
    let actionCell = document.createElement("td");

    // Adding values to cells
    nameCell.textContent = name;
    rollCell.textContent = roll;
    courseCell.textContent = course;

    // Creating Delete button
    let deleteButton = document.createElement("button");

    deleteButton.textContent = "Delete";
    deleteButton.className = "deleteBtn";

    // Delete button event
    deleteButton.addEventListener("click", function() {
        row.remove();
        message.textContent = "Student deleted successfully";
    });

    // Adding delete button to action cell
    actionCell.appendChild(deleteButton);

    // Adding cells to row
    row.appendChild(nameCell);
    row.appendChild(rollCell);
    row.appendChild(courseCell);
    row.appendChild(actionCell);

    // Adding row to table
    studentList.appendChild(row);

    // Display success message
    message.textContent = "Student added successfully";

    // Clear input fields
    nameInput.value = "";
    rollInput.value = "";
    courseInput.value = "";
});

```

7. CODE EXPLANATION

Code / Concept	Explanation
<!DOCTYPE html>	Declares the document as an HTML5 document.
<link rel="stylesheet" ...>	Connects the external CSS file with the HTML document.
<div class="container">	Wraps and centers the entire page content.
<form id="studentForm">	Defines the student registration form.
<input type="text" id="name">	Accepts the student's name as text input.
<input type="number" id="roll">	Accepts the student's roll number as a numeric input.
<select id="course">	Provides a dropdown list of available courses.
<button type="submit">	Submits the form and triggers the JavaScript submit handler.
<p id="message">	Displays dynamic validation or status messages.
<table id="studentTable">	Displays the list of registered students.
<tbody id="studentList">	Acts as the container into which new rows are inserted dynamically.
getElementById()	Selects a specific HTML element using its id attribute.
addEventListener("submit", fn)	Registers a function to run when the form is submitted.
event.preventDefault()	Stops the browser's default page-reload behaviour on form submission.
.value	Reads the current value entered in an input or select field.
if (name === "" ...)	Validates that the required fields are not empty.
document.createElement("tr"/>"td"/"button")	Dynamically creates new DOM elements at runtime.
.textContent	Sets the visible text content of a newly created element.
.className	Assigns a CSS class (deleteBtn) to the Delete button for styling.
addEventListener("click", fn) (delete button)	Registers a click handler that deletes that specific row.
row.remove()	Removes the corresponding table row from the DOM.
appendChild()	Attaches a newly created child element to its parent element.
.value = ""	Clears an input field after successful submission.

8. TEST CASES

The webpage was opened in a web browser and tested for form validation, dynamic row addition, dynamic row deletion, and correctness of the displayed data.

Test Case	Operation	Expected Result	Actual Result	Status
TC-01	Open index.html	Registration form and an empty student table should be displayed	Displayed successfully	Pass
TC-02	Submit form with all fields empty	"Please fill all fields" message should be displayed	Message displayed	Pass
TC-03	Enter name only, leave Roll Number and Course empty, submit	Validation message should be displayed	Message displayed	Pass
TC-04	Enter valid name, roll number, and select a course, then submit	Student should be added as a new row in the table	Row added, "Student added successfully" displayed	Pass
TC-05	Add a second student with different details	New row appended below the first	Row added successfully	Pass

Test Case	Operation	Expected Result	Actual Result	Status
TC-06	Check input fields after successful submission	Fields should be cleared	Fields cleared	Pass
TC-07	Add a third student, then click Delete on the middle row	That specific row should be removed; other rows remain unchanged	Row removed, "Student deleted successfully" displayed	Pass
TC-08	Select each option in the Course dropdown	Options (JavaScript, Python, Java) should be selectable	Options selected successfully	Pass
TC-09	Enter non-numeric text in the Roll Number field	Browser should restrict the field to numeric values	Number input validated by browser	Pass
TC-10	Add multiple students and verify table contents	All added rows should display the correct name, roll number, and course	Correct data displayed in every row	Pass

9. OUTPUT

The webpage was executed in a web browser (Chromium). Screenshots of the actual page behaviour for the key test cases are shown below.

Output 1 (TC-01): Initial page load — empty form and empty table

The screenshot displays a web application titled "Student Registration Form". It features three input fields: "Student Name:" with a placeholder "Enter student name", "Roll Number:" with a placeholder "Enter roll number", and "Course:" with a dropdown menu showing "Select Course". Below these fields is an "Add Student" button. Underneath the button is a section titled "Student List" containing a table with four columns: "Name", "Roll Number", "Course", and "Action". The table is currently empty.

Output 2 (TC-02): Submit with all fields empty

Student Registration Form

Student Name:

Roll Number:

Course:

Please fill all fields

Student List

Name	Roll Number	Course	Action
------	-------------	--------	--------

Output 3 (TC-03): Submit with only Name filled

Student Registration Form

Student Name:

Roll Number:

Course:

Please fill all fields

Student List

Name	Roll Number	Course	Action
------	-------------	--------	--------

Output 4 (TC-04): First student added successfully

Student Registration Form

Student Name:

Roll Number:

Course:

Add Student

Student added successfully

Student List

Name	Roll Number	Course	Action
Rahul Kumar	101	JavaScript	Delete

Output 5 (TC-05): Second student added

Student Registration Form

Student Name:

Roll Number:

Course:

Add Student

Student added successfully

Student List

Name	Roll Number	Course	Action
Rahul Kumar	101	JavaScript	Delete
Sneha Reddy	102	Python	Delete

Output 6: Third student added (for delete test)

Student Registration Form

Student Name:

Roll Number:

Course:

Add Student

Student added successfully

Student List

Name	Roll Number	Course	Action
Rahul Kumar	101	JavaScript	Delete
Sneha Reddy	102	Python	Delete

Output 7 (TC-07): Middle row deleted — remaining rows intact

Student Registration Form

Student Name:

Roll Number:

Course:

Add Student

Student deleted successfully

Student List

Name	Roll Number	Course	Action
Rahul Kumar	101	JavaScript	Delete
Arjun Verma	103	Java	Delete

10. RESULT

Thus, a dynamic Student Registration Form was successfully designed and developed using HTML, CSS, and JavaScript. The form validates required fields before submission, and JavaScript DOM manipulation methods such as `createElement()`, `appendChild()`, `textContent`, and `remove()` were used to dynamically add and delete student records in the table without reloading the page. Event handling was implemented using `addEventListener()` for both the form submission and the individual Delete buttons. The webpage was executed and verified successfully in a web browser, with all test cases producing the expected results.